

作品名:

物理エンジン: 分離軸定理(SAT)による多角形衝突の実装
(Physics Engine Demo: Implementation of Polygon Collision via SAT)

作成期間:

2025 年 8 月 (約 1 週間)

プログラム作品の開発環境についてご記入ください

・動作確認した OS 名とそのバージョン

Windows 11 Home / Professional 64bit

・開発に使用したライブラリ、ミドルウェアとそのバージョン

Win32 API (GDI), Standard Template Library (STL)

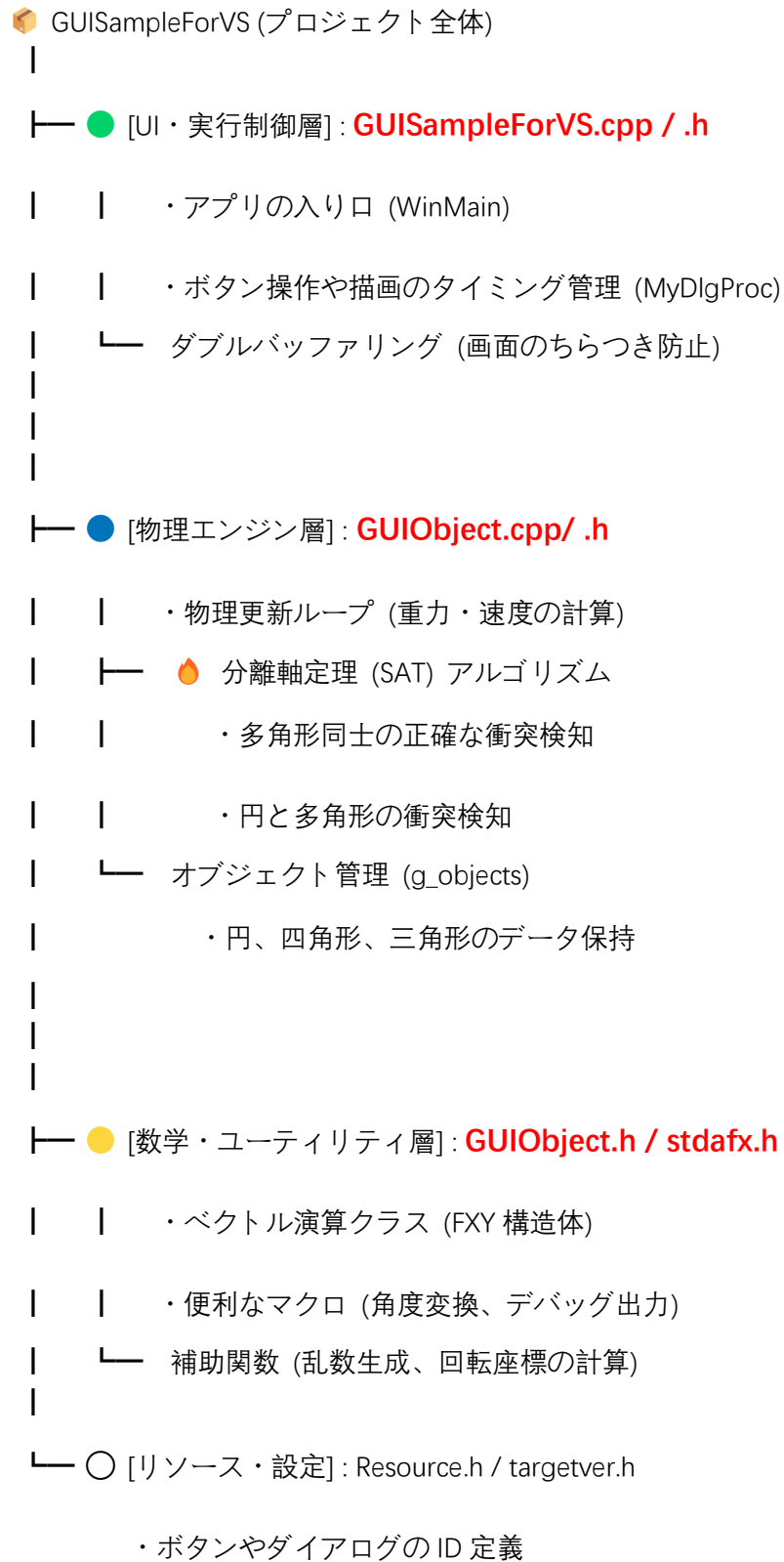
・開発に使用したエンジンのバージョン

なし

・開発に使用した開発環境のバージョン

Visual Studio 2022 Version 17.0 以上

プロジェクト構造図：



どうしてそのプログラムを作ろうと思いましたか？

ゲームプログラミングにおいて、キャラクターやオブジェクトの「手触り」は非常に重要です。

従来の AABB（軸平行境界ボックス）では回転する多角形の衝突を正確に表現できず、不自然な隙間や埋まりが発生してしまいます。そこで、より高度な「分離軸定理（SAT）」を実装することで、三角形や四角形がどの角度でぶつかっても正確に反応する、ストレスのない物理挙動を実現したいと考え作成しました。

プログラムを作成する上で注意した事は？

数学的な正確さと、計算負荷のバランスに注意しました。特に円と多角形の衝突判定では、ポリゴンの各辺の法線だけでなく、円の中心に最も近い「頂点」へのベクトルも分離軸の候補としてテストする必要がある、判定漏れがないようロジックを慎重に構築しました。また、浮動小数点の計算誤差による「オブジェクトの埋まり」を防ぐため、Positional Correction（位置補正）と Slop（許容誤差）を導入し、安定性を高めました。

プログラムを作成する上で大変だった所は？

三角形の衝突解決です。

重心計算に基づいたローカル座標の設定や、回転行列を用いたワールド座標への変換など、ベクトル演算のデバッグに時間を費やしました。特に SAT 実装初期には、法線ベクトルの向き（obj1 から obj2 へ方向）が反転してしまい、オブジェクト同士が引き合ってしまう現象が発生したため、内積を用いた方向補正ロジックを組み込むことで解決しました。

力をいれて作った部分で、「プログラム上」で特に注意してみても らいたい所は？

GUIObject.cpp 内

1. 汎用的な SAT の実装: PolygonCollision 関数

- 実装箇所: GUIObject.cpp の 111 行目～155 行目付近
- ここが凄い: obj1.vertex_count と obj2.vertex_count を使用し、ループ内で辺の法線（分離軸）を動的に生成して投影（ProjectPolygon）を行っています。これにより、三角形（頂点 3）と四角形（頂点 4）が混在していても、共通のロジックでピクセル精度の衝突判定ができています。

2. 円と多角形の衝突: CirclePolygonCollision 関数

厳密な 2 段階の判定が行われています。

- **実装箇所:** GUIObject.cpp の 158 行目〜223 行目付近
- **ここが凄い:**
 1. まず多角形の「辺の法線」方向に影を落として判定します。
 2. さらに、円に最も近い「頂点」への方向も軸としてテストしています。
 - これにより、円が多角形の「カド」にぶつかった際の挙動が非常にスムーズになります。

3. トンネル現象の抑制: SUB_STEPS を用いたサブステップ処理

高速に動く物体が壁をすり抜ける問題への対策です。

- **実装箇所:** GUIObject.cpp の 288 行目〜381 行目 (UpdateObject 関数内)
- **ここが凄い:** 物理定数 PhysicsConstants::SUB_STEPS (値は 10) を使用し、1 フレームの間に衝突判定と位置補正を 10 回に分けて計算しています。これにより、計算負荷を抑えつつ、高速移動時の突き抜けを劇的に減らしています。

4. リアルな反発: インパルス応答の実装

単純な速度反転ではなく、物理学に基づいたエネルギー保存が考慮されています。

- **実装箇所:** GUIObject.cpp の 366 行目〜380 行目付近
- **ここが凄い:**
 - 2 つの物体の「反発係数 (restitution)」のうち、小さい方を採用する (std::min) といった細かい配慮があります。
 - 質量 (mass) の逆数 (inv_mass) を用いて運動量の変化 (インパルス) を計算し、速度を更新しています。
 - さらに、POSITIONAL_CORRECTION_PERCENT を用いて、わずかなめり込みを解消する補正処理も入っており、物体が重なって固まるのを防いでいます。